Théorie des langages

Automates à états finis

2ème Année Informatique Appliquée

Pro. Mohamed Nazih OMRI

Aujourd'hui

Rappels

Déterminisation

Automates et expressions régulières

Langages non-réguliers

Propriétés de clôture

Le mot de la fin

Déterminisation

- Théorème: Tout *NFA* peut être simulé par un *DFA*.

Déterminisation

- Théorème: Tout *NFA* peut être simulé par un *DFA*.
- Preuve:
  - Soit $(Q, \Sigma, \delta, q_0, F)$ un *NFA*.
  - Il reconnaît le même langage que le *DFA*

    $$(Q', \Sigma, \delta', \{q_0\}, F')$$

    avec:

    - $Q' = \mathcal{P}(Q)$: Q' est constitué de tous les sous-ensembles de Q.
    - F' est constitué de tous les sous-ensembles de Q avec au moins un élément commun avec F.
    - Pour tout $S \in Q'$, et pour tout $a \in \Sigma$,

      $$\delta'(S, a) = \cup_{q \in S} \delta(q, a).$$

Déterminisation

- Théorème: Tout *NFA* peut être simulé par un *DFA*.
- Preuve:
  - Soit $(Q, \Sigma, \delta, q_0, F)$ un *NFA*.
  - Il reconnaît le même langage que le *DFA*

    $$(Q', \Sigma, \delta', \{q_0\}, F')$$

    avec:

    - $Q' = \mathcal{P}(Q)$: Q' est constitué de tous les sous-ensembles de Q.
    - F' est constitué de tous les sous-ensembles de Q avec au moins un élément commun avec F.
    - Pour tout $S \in Q'$, et pour tout $a \in \Sigma$,

      $$\delta'(S, a) = \cup_{q \in S} \delta(q, a).$$

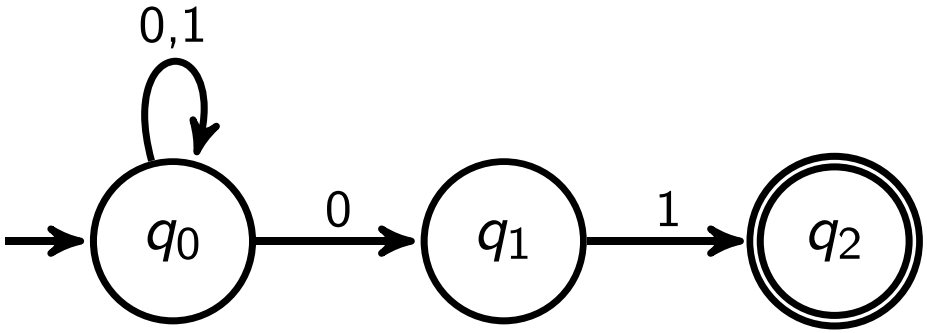
  - Pourquoi: il est fait exactement pour que

    $$\hat{\delta}'(q_0, w) = \hat{\delta}(\{q_0\}, w)$$

    pour tout mot w.

Exemple 1

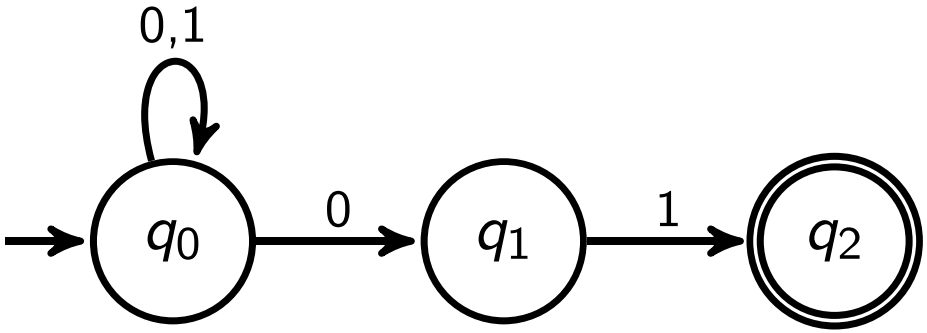
- Un *NFA*:



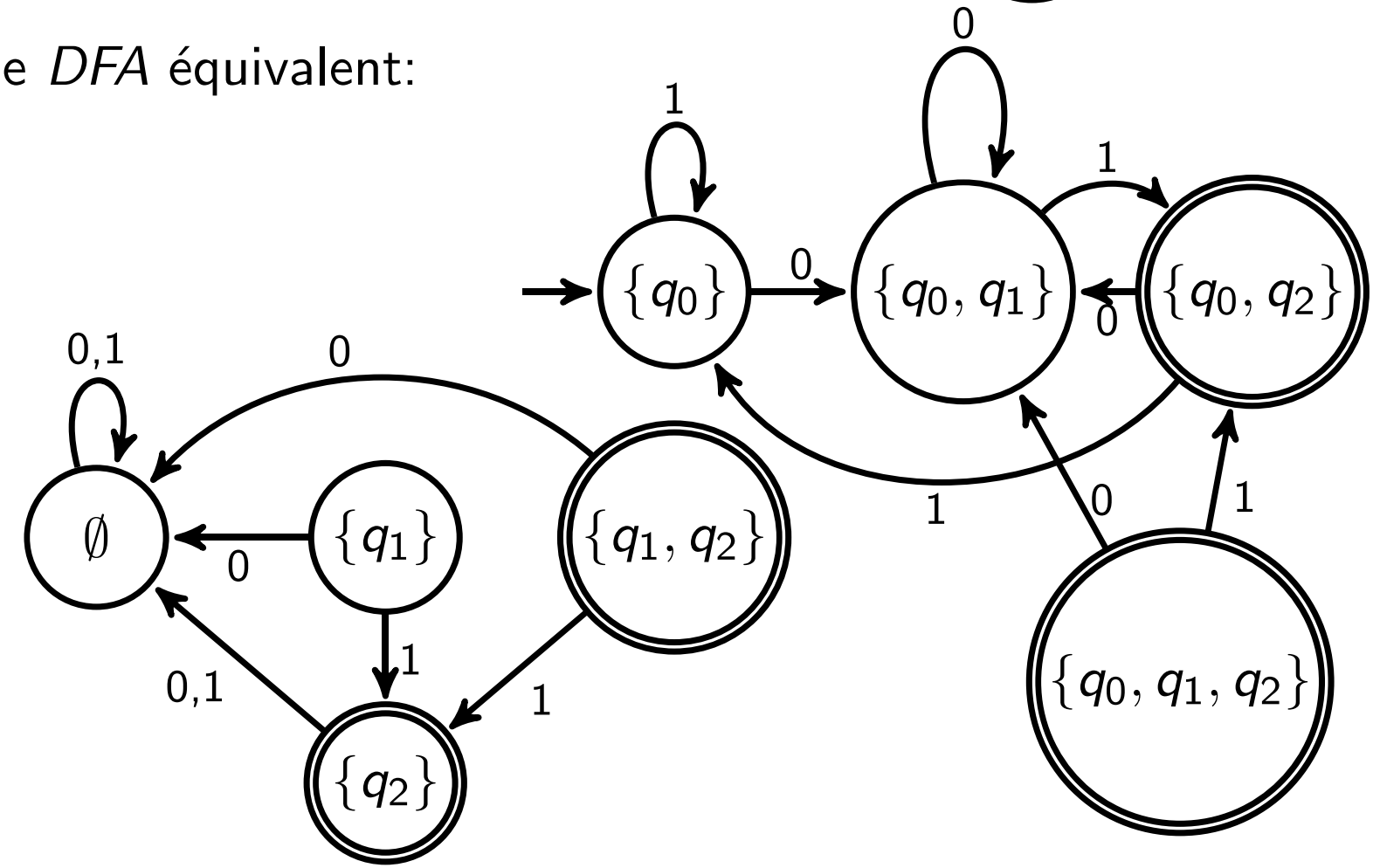
- Le *DFA* équivalent:

Exemple 1

- Un *NFA*:

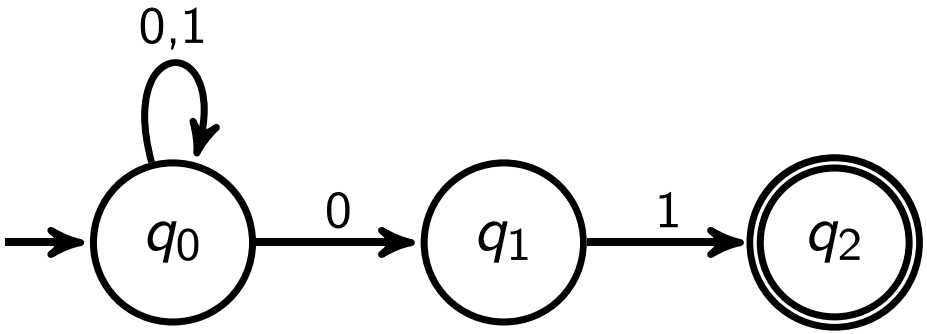


- Le *DFA* équivalent:

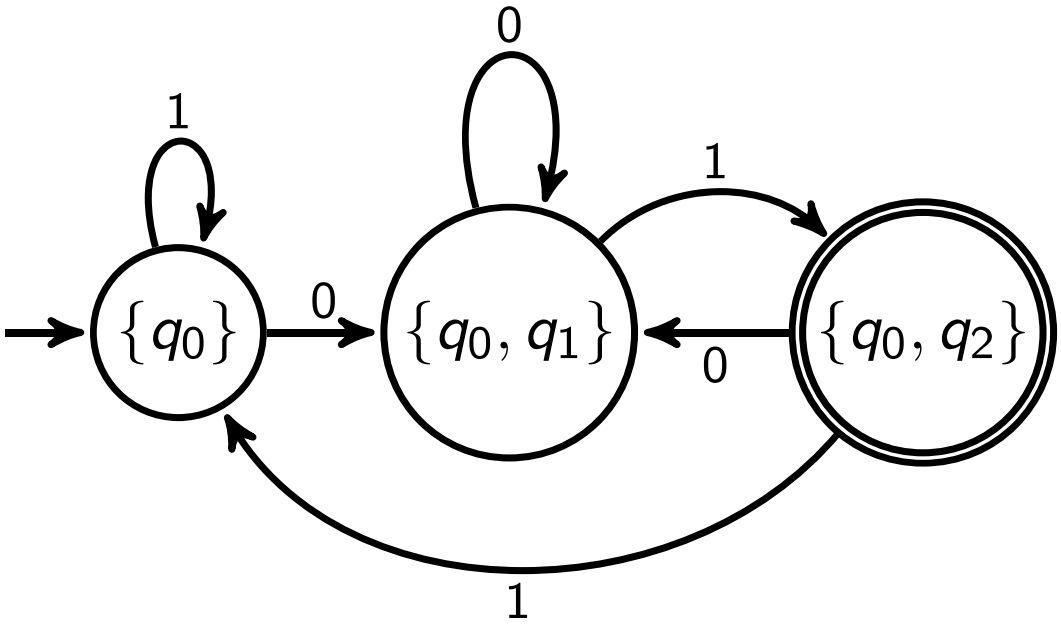


Exemple 1: suite

- Le *DFA*:

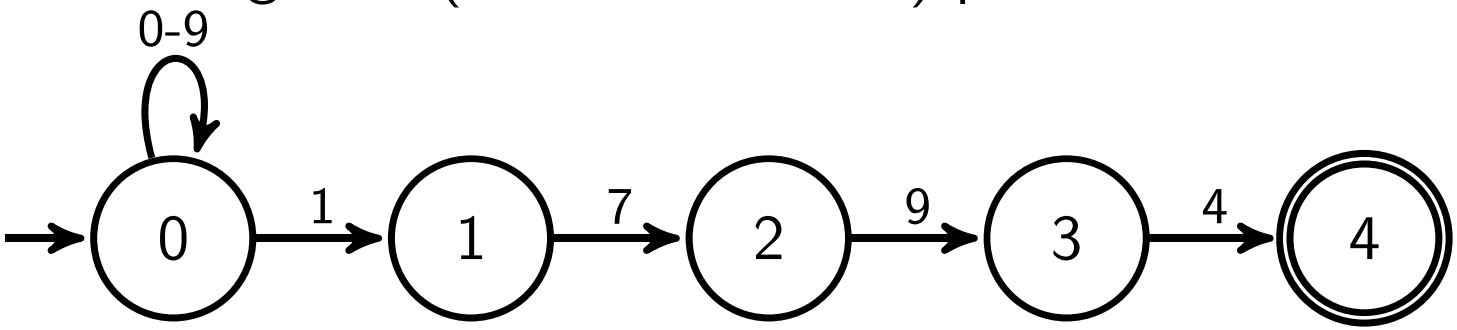


- La partie atteignable du *DFA* équivalent:



Exemple: un digicode

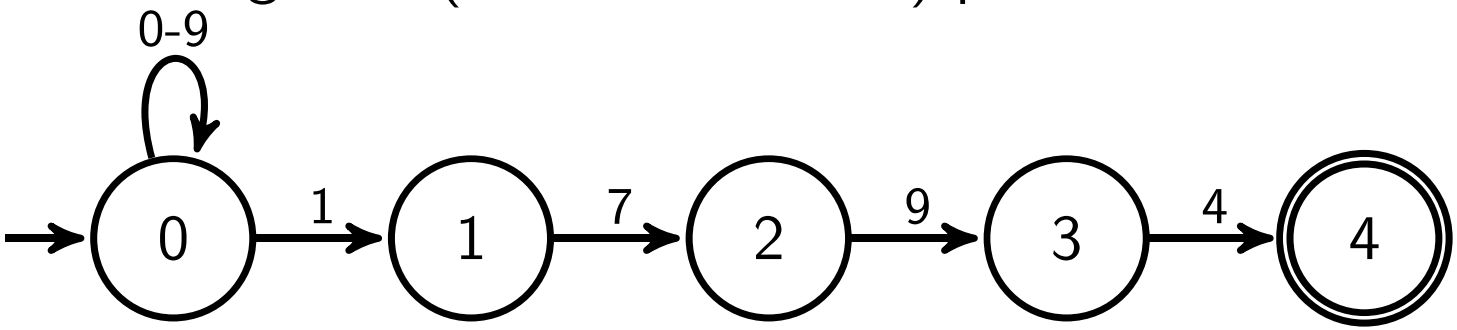
- Réaliser un digicode (code secret 1794) par un *NFA*:



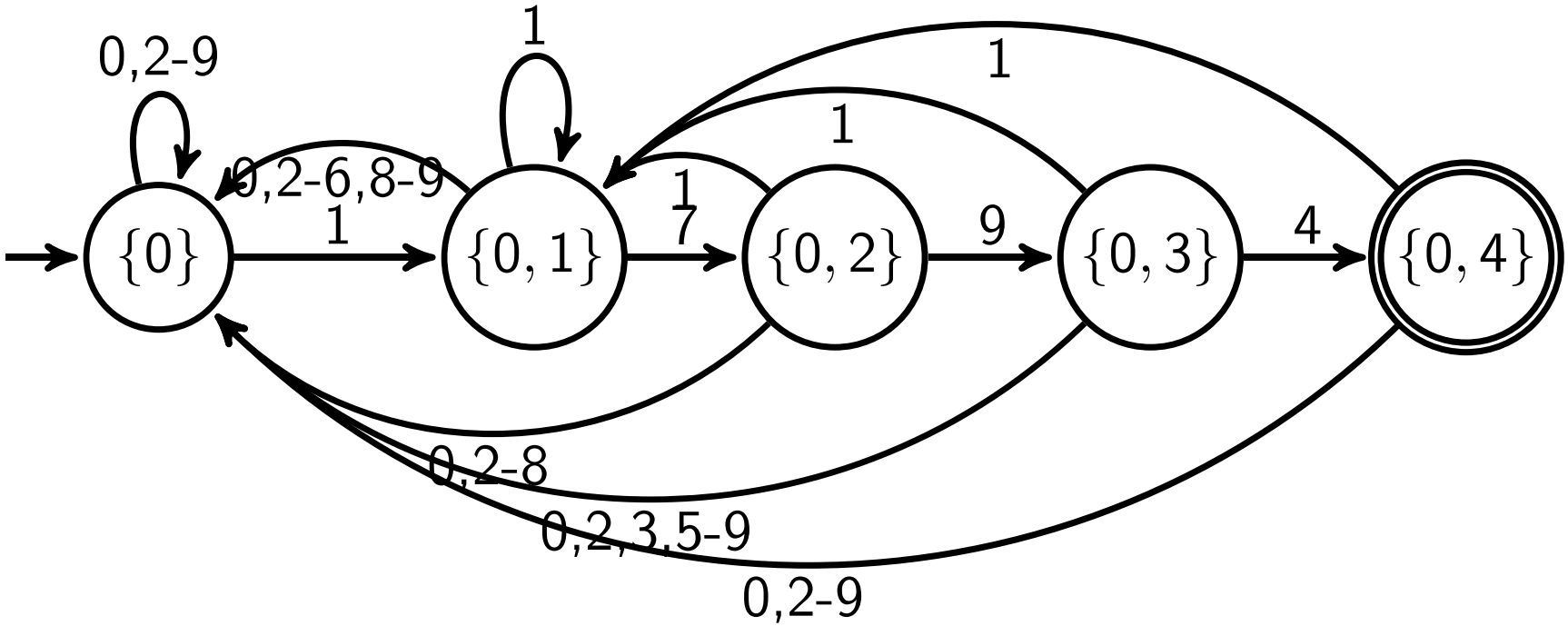
- Le *DFA* obtenu (circuit électronique correspondant):

Exemple: un digicode

- Réaliser un digicode (code secret 1794) par un *NFA*:

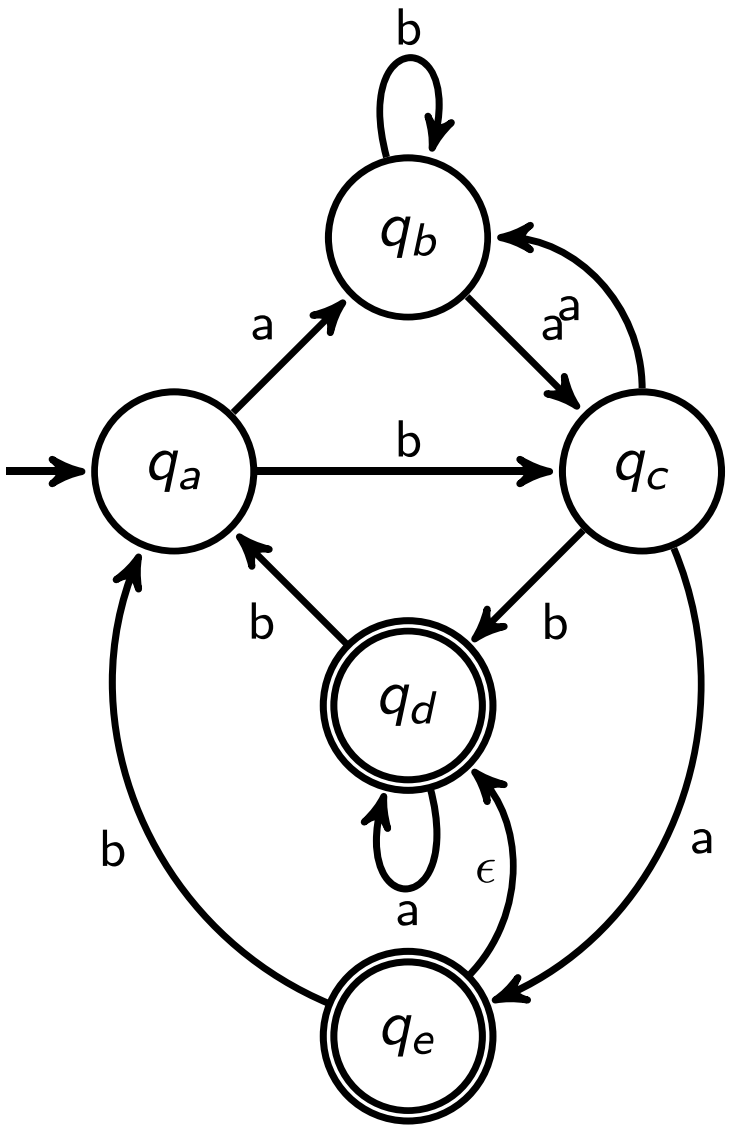


- Le *DFA* obtenu (circuit électronique correspondant):



- Tous les autres sous-ensembles de $\{0, 1, 2, 3, 4\}$ ne sont pas atteignables: ils ne sont pas représentés ici.

Encore une extension: automate fini non-déterministe avec ϵ-transitions ($NFA - \epsilon$)



- Le principe:
  1. on autorise toujours plusieurs transitions avec le même symbole. (non-déterminisme)
  2. mais aussi les transitions "spontanées":
     - certaines transitions peuvent être étiquetées par le mot vide ϵ.
     - Elles peuvent être prises sans lire de lettre.
- Exemple: le mot *baaaaa* est accepté.

$NFA - \epsilon$: formalisations

- Formalisation de la notion d'automate:

- Formalisation de la notion de mot reconnu:

$NFA - \epsilon$: formalisations

- Formalisation de la notion d'automate:
  - ... $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$ ...
- Formalisation de la notion de mot reconnu:

$NFA - \epsilon$: formalisations

- Formalisation de la notion d'automate:
  - ... $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to \mathcal{P}(Q)$ ...
- Formalisation de la notion de mot reconnu:
  - ... $\hat{\delta}(q, wa) = \bigcup_{p \in \hat{\delta}(q,w)} \epsilon - \text{clôture}(\delta(p, a))$.

$$\epsilon\text{−clôture}(q) = \left\{ \begin{array}{l} \text{tous les états joignables à partir de } q \\ \text{par une suite de } n \geq 0 \text{ transitions} \\ \text{étiquetées par } \epsilon \end{array} \right\} \ldots$$

$NFA - \epsilon$: formalisations

- Formalisation de la notion d'automate:
  - ... $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$ ...
- Formalisation de la notion de mot reconnu:
  - ... $\hat{\delta}(q, wa) = \bigcup_{p \in \hat{\delta}(q,w)} \epsilon - \text{clôture}(\delta(p, a))$.

$$\epsilon\text{−clôture}(q) = \left\{ \begin{array}{l} \text{tous les états joignables à partir de } q \\ \text{par une suite de } n \geq 0 \text{ transitions} \\ \text{étiquetées par } \epsilon \end{array} \right\} \ldots$$

- Tout *DFA* peut être vu comme un *NFA*, tout *NFA* peut être vu comme un $NFA - \epsilon$.

$NFA - \epsilon$: formalisations

- Formalisation de la notion d'automate:
  - ... $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$ ...
- Formalisation de la notion de mot reconnu:
  - ... $\hat{\delta}(q, wa) = \bigcup_{p \in \hat{\delta}(q,w)} \epsilon - \text{clôture}(\delta(p, a))$.

$$\epsilon\text{−clôture}(q) = \left\{ \begin{array}{l} \text{tous les états joignables à partir de } q \\ \text{par une suite de } n \geq 0 \text{ transitions} \\ \text{étiquetées par } \epsilon \end{array} \right\} \ldots$$

- Tout *DFA* peut être vu comme un *NFA*, tout *NFA* peut être vu comme un $NFA - \epsilon$.
- Théorème: Tout $NFA - \epsilon$ peut être simulé par un *DFA*.

$NFA - \epsilon$: formalisations

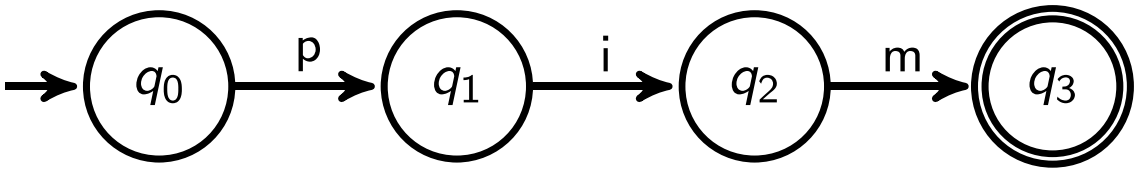
- Formalisation de la notion d'automate:
  - ... $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to \mathcal{P}(Q)$ ...
- Formalisation de la notion de mot reconnu:
  - ... $\hat{\delta}(q, wa) = \bigcup_{p \in \hat{\delta}(q,w)} \epsilon - \text{clôture}(\delta(p, a))$.

$$\epsilon\text{−clôture}(q) = \left\{ \begin{array}{l} \text{tous les états joignables à partir de } q \\ \text{par une suite de } n \geq 0 \text{ transitions} \\ \text{étiquetées par } \epsilon \end{array} \right\} \ldots$$

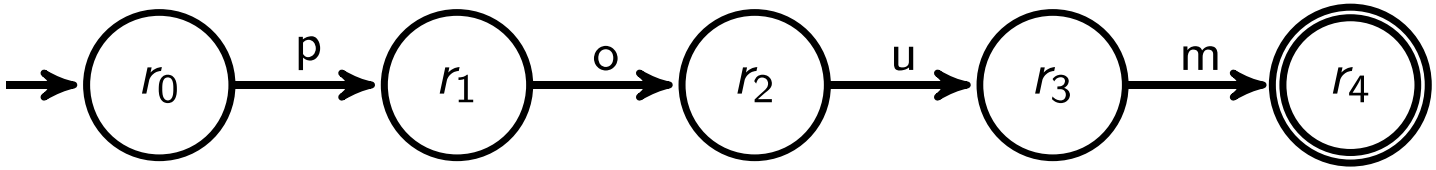
- Tout *DFA* peut être vu comme un *NFA*, tout *NFA* peut être vu comme un $NFA - \epsilon$.
- Théorème: Tout $NFA - \epsilon$ peut être simulé par un *DFA*.
  - ... même principe ...

Un intérêt: permettre de combiner directement des automates.

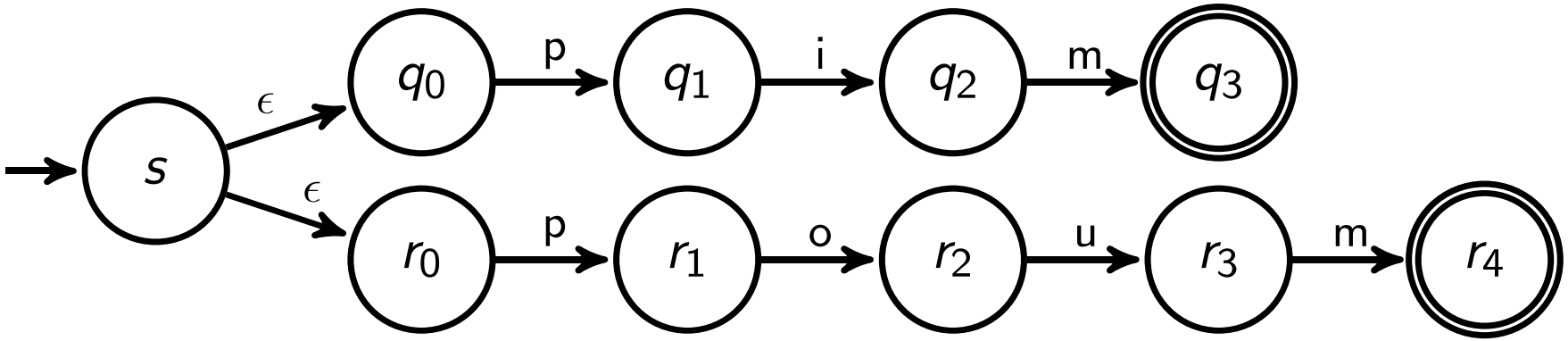
- Exemple:
  - à partir d'un automate qui reconnaît *pim*



  - et d'un automate qui reconnaît *poum*



  - construire un automate qui reconnaît *pim*|*poum* à coup de copier coller:



Aujourd'hui

Rappels

Déterminisation

Automates et expressions régulières

Langages non-réguliers

Propriétés de clôture

Le mot de la fin

Liens avec les expressions régulières

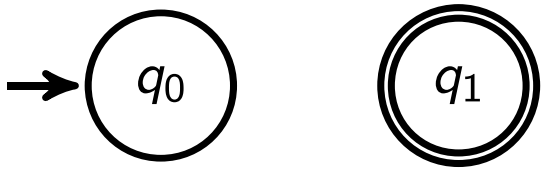
- Théorème de Kleene:

$$L(DFA) = L(NFA) = L(NFA - \epsilon) = \text{Langages réguliers.}$$

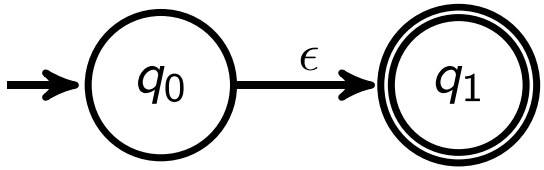
- Autrement dit: les assertions suivantes sont équivalentes.
  - L est accepté par un DFA.
  - L est accepté par un NFA.
  - L est accepté par un $NFA - \epsilon$.
  - L est régulier: $L = [\![p]\!]$ pour un motif p.

Des expressions régulières aux automates: motifs de base

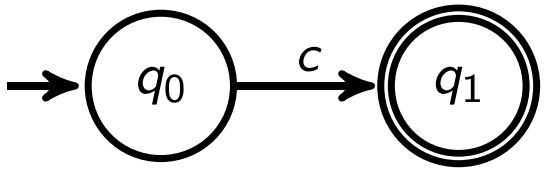
- Étant donné un motif p, on peut construire un automate qui reconnaît $[\![p]\!]$.
  - Pour le motif $\emptyset$:



  - Pour le motif ϵ:

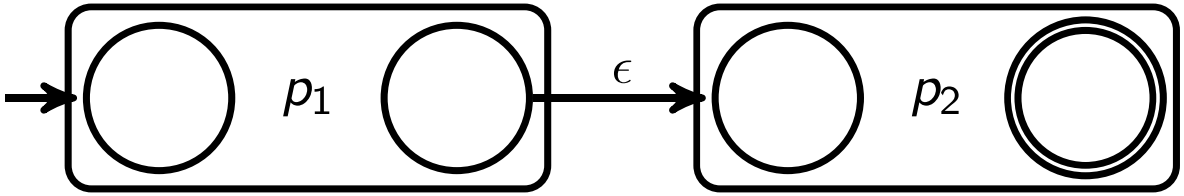


  - Pour le motif c, $c \in \Sigma$:

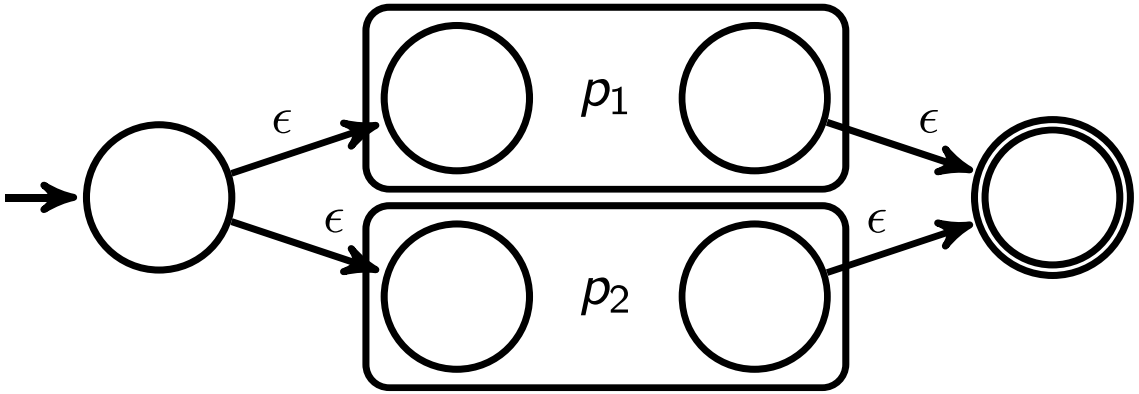


Des expressions régulières aux automates: autres motifs

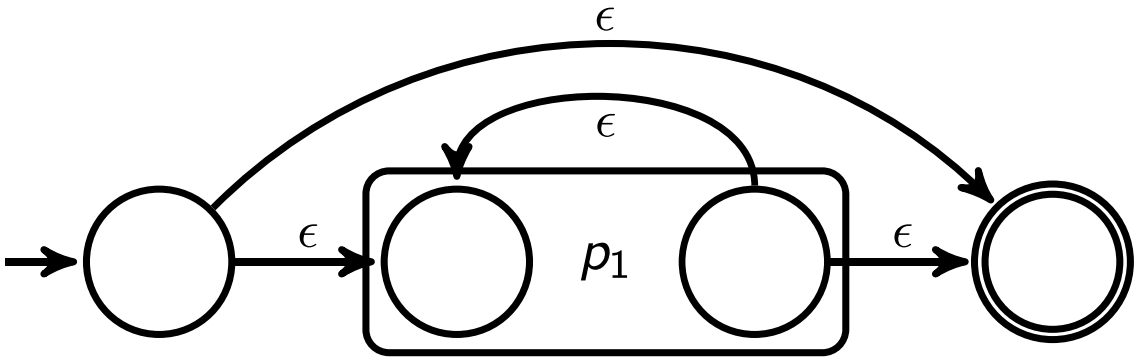
- Inductivement, si les motifs p_1 et p_2 correspondent à des automates,
  - Pour le motif $p_1.p_2$:



  - Pour le motif $p_1|p_2$:



  - Pour le motif p_1*:



Des automates aux expressions régulières: un peu d'algèbre

- Les identités $(L_1 \cup L_2)L = L_1.L \cup L_2.L$ et $L.(L_1 \cup L_2) = L.L_1 \cup L.L_2$ incitent à aussi noter $+$ la disjonction:

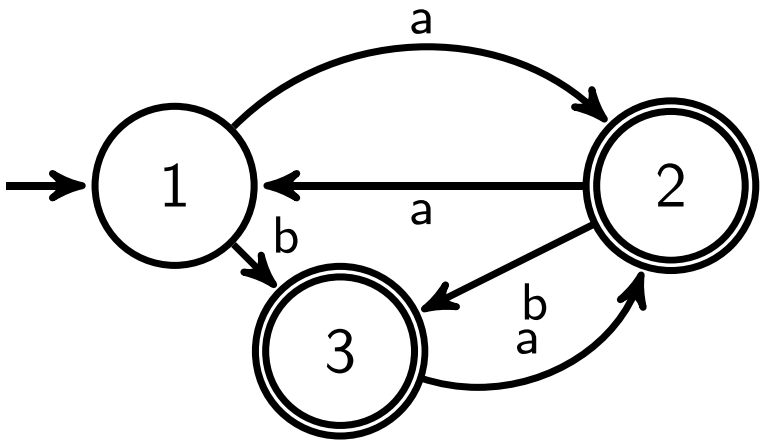
$$\begin{array}{rcl} (L_1 + L_2)L & = & L_1L + L_2L \\ L(L_1 + L_2) & = & LL_1 + LL_2 \end{array}$$

- Les identités $\emptyset \cup L = L \cup \emptyset = L$ et $\{\epsilon\}L = L\{\epsilon\} = L$ incitent à aussi noter 0 pour $\emptyset$, et 1 pour $\{\epsilon\}$:

$$\begin{array}{rcl} L + 0 & = & L \\ 0 + L & = & L \\ 1.L & = & L \\ L.1 & = & L \end{array}$$

Des automates aux expressions régulières: un peu d'algèbre

- Passer d'un automate à un langage...



- ...revient à résoudre un système d'équations.

$$\begin{cases} X_1 & = & aX_2 + bX_3 \\ X_2 & = & aX_1 + bX_3 + 1 \\ X_3 & = & aX_2 + 1 \end{cases}$$

- Cas général:
  - X_q est le langage constitué des mots w tel que $\hat{\delta}(q, w) \in F$.
  - $X_q = \sum_p K_{p,q} X_p + L_q$ avec
    - $K_{p,q} = \{a \in \Sigma | q \in \delta(p, a)\}$,
    - L_q vaut 0 si $q \notin F$, $\{1\}$ sinon.

Des automates aux expressions régulières: résoudre des équations

- Si K ne contient pas le mot vide, l'équation

$$X = KX + L$$

admet comme unique solution $X = K^*L$.

Des automates aux expressions régulières: résoudre des équations

- Si K ne contient pas le mot vide, l'équation

$$X = KX + L$$

admet comme unique solution $X = K^*L$.

- Preuve:
  - K^*L est bien solution, car

$$K(K^*L) + L = (KK*)L + L = (KK^* + 1)L = K^*L$$

Des automates aux expressions régulières: résoudre des équations

- Si K ne contient pas le mot vide, l'équation

$$X = KX + L$$

admet comme unique solution $X = K^*L$.

- Preuve:
  - K^*L est bien solution, car

$$K(K^*L) + L = (KK*)L + L = (KK^* + 1)L = K^*L$$

  - Soit Y une autre solution, prouvons que $X = Y$, pour $X = K^*L$.
    - On prouve par récurrence sur $|w|$ que $w \in K^*L$ implique $w \in Y$:
      Si $|w| = 0$, $w \in L$, car $\epsilon \notin K$. Donc $w \in KY + L = Y$.
      Si $|w| = n + 1$. Si $w \in L$, alors $w \in KY + L = Y$. Sinon,
      $w = w'w''w'''$, avec $w' \in K$, $w'' \in K^*$, $w''' \in L$. Puisque $w' \neq \epsilon$,
      $|w''w'''| \leq n$ et par hypothèse de récurrence $w''w''' \in Y$, et donc
      $w = w'w''w''' \in KY \subset KY + L = Y$.
    - Donc $X \subset Y$.
    - De facon duale, $Y \subset X$.

Des automates aux expressions régulières: résoudre des équations

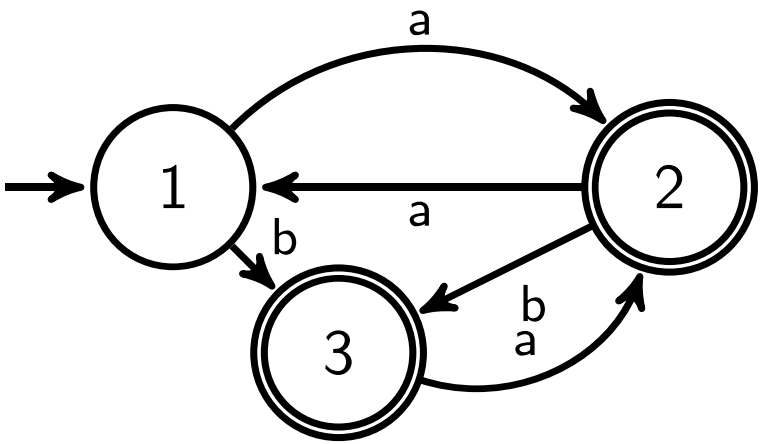
- Si K ne contient pas le mot vide, l'équation

$$X = KX + L$$

admet comme unique solution $X = K^*L$.

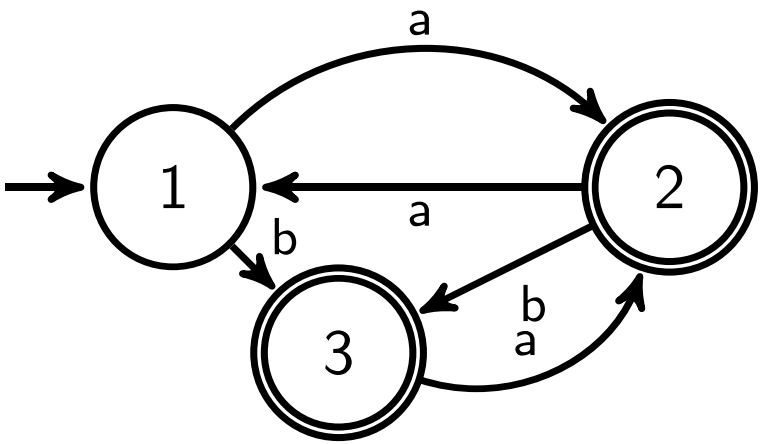
- Et donc si K et L sont réguliers, X aussi.

Des automates aux expressions régulières: exemple



$$\begin{cases} X_1 & = & aX_2 + bX_3 \\ X_2 & = & aX_1 + bX_3 + 1 \\ X_3 & = & aX_2 + 1 \end{cases}$$

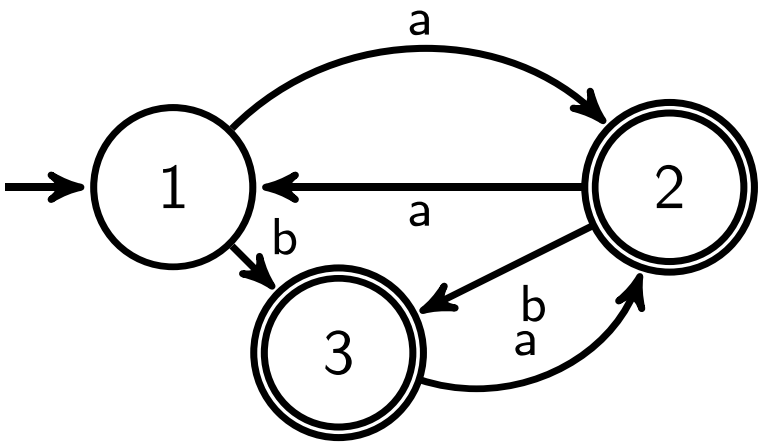
Des automates aux expressions régulières: exemple



$$\begin{cases} X_1 & = & aX_2 + bX_3 \\ X_2 & = & aX_1 + bX_3 + 1 \\ X_3 & = & aX_2 + 1 \end{cases}$$

- Remplaçons X_3 par $aX_2 + 1$.

Des automates aux expressions régulières: exemple

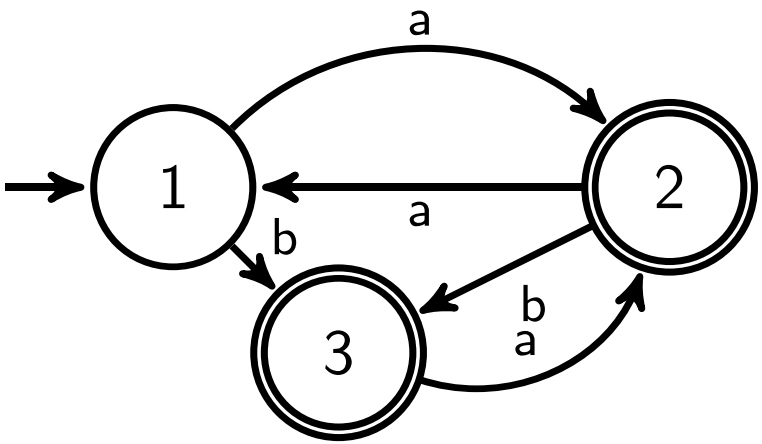


$$\begin{cases} X_1 & = & aX_2 + bX_3 \\ X_2 & = & aX_1 + bX_3 + 1 \\ X_3 & = & aX_2 + 1 \end{cases}$$

$$\begin{cases} X_1 & = & aX_2 + b(aX_2 + 1) \\ X_2 & = & aX_1 + b(aX_2 + 1) + 1 \\ X_3 & = & aX_2 + 1 \end{cases}$$

- Remplaçons X_3 par $aX_2 + 1$.

Des automates aux expressions régulières: exemple

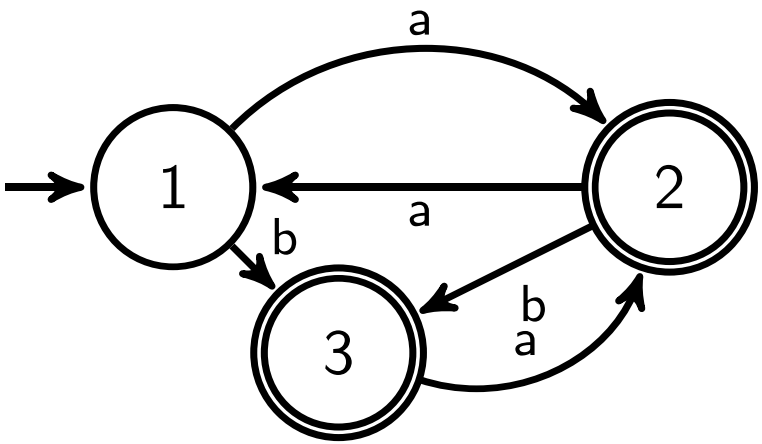


$$\begin{cases} X_1 & = & aX_2 + bX_3 \\ X_2 & = & aX_1 + bX_3 + 1 \\ X_3 & = & aX_2 + 1 \end{cases}$$

$$\begin{cases} X_1 & = & (a + ba)X_2 + b \\ X_2 & = & aX_1 + baX_2 + (b + 1) \\ X_3 & = & aX_2 + 1 \end{cases}$$

- Remplaçons X_3 par $aX_2 + 1$.
- Remplaçons X_1 par $(a + ba)X_2 + b$.

Des automates aux expressions régulières: exemple

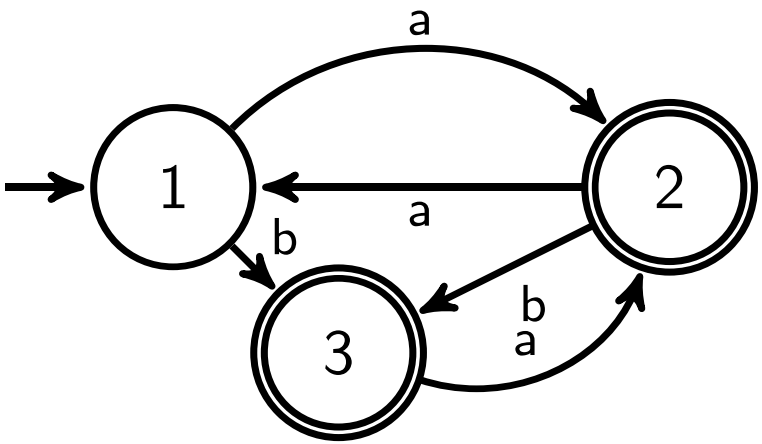


$$\begin{cases} X_1 & = & aX_2 + bX_3 \\ X_2 & = & aX_1 + bX_3 + 1 \\ X_3 & = & aX_2 + 1 \end{cases}$$

$$\begin{cases} X_2 & = & a((a+ba)X_2 + b) + baX_2 + (b+1) \\ X_1 & = & (a+ba)X_2 + b \\ X_3 & = & aX_2 + 1 \end{cases}$$

- Remplaçons X_3 par $aX_2 + 1$.
- Remplaçons X_1 par $(a+ba)X_2 + b$.

Des automates aux expressions régulières: exemple

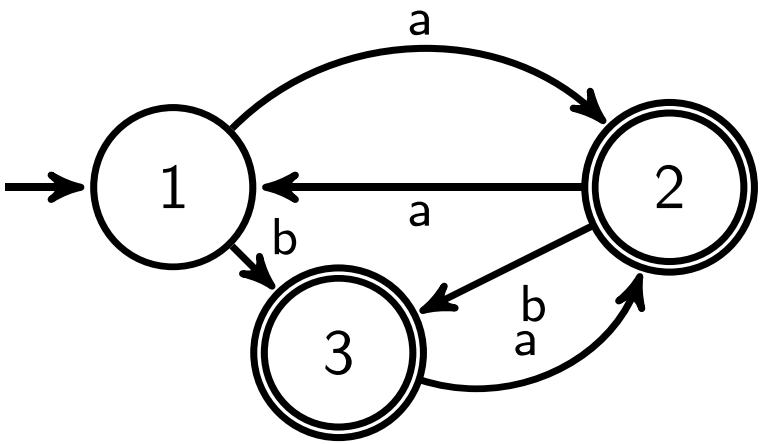


$$\begin{cases} X_1 & = & aX_2 + bX_3 \\ X_2 & = & aX_1 + bX_3 + 1 \\ X_3 & = & aX_2 + 1 \end{cases}$$

$$\begin{cases} X_2 & = & (aa + aba + ba)X_2 + (ab + b + 1) \\ X_1 & = & (a + ba)X_2 + b \\ X_3 & = & aX_2 + 1 \end{cases}$$

- Remplaçons X_3 par $aX_2 + 1$.
- Remplaçons X_1 par $(a + ba)X_2 + b$.
- Résolvons l'équation $X_2 = (aa + aba + ba)X_2 + (ab + b + 1)$.

Des automates aux expressions régulières: exemple

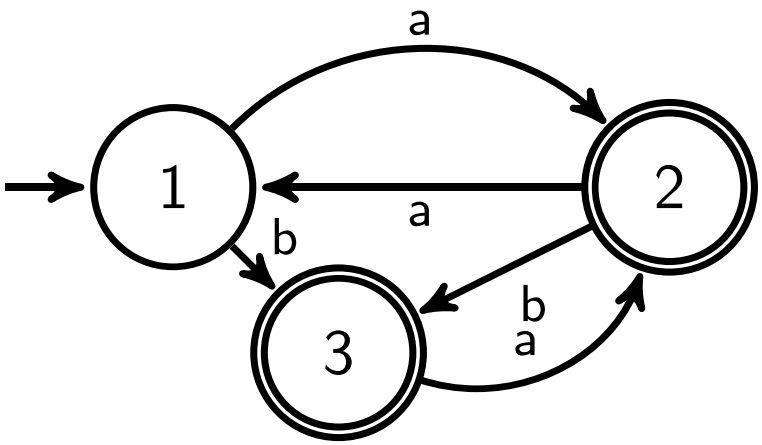


$$\begin{cases} X_1 &= aX_2 + bX_3 \\ X_2 &= aX_1 + bX_3 + 1 \\ X_3 &= aX_2 + 1 \end{cases}$$

$$\begin{cases} X_2 &= (aa + aba + ba)^*(ab + b + 1) \\ X_1 &= (a + ba)X_2 + b \\ X_3 &= aX_2 + 1 \end{cases}$$

- Remplaçons X_3 par $aX_2 + 1$.
- Remplaçons X_1 par $(a + ba)X_2 + b$.
- Résolvons l'équation $X_2 = (aa + aba + ba)X_2 + (ab + b + 1)$.
- Reportons.

Des automates aux expressions régulières: exemple



$$\begin{cases} X_1 &= aX_2 + bX_3 \\ X_2 &= aX_1 + bX_3 + 1 \\ X_3 &= aX_2 + 1 \end{cases}$$

$$\begin{cases} X_2 &= (aa + aba + ba)^*(ab + b + 1) \\ X_1 &= (a + ba)(aa + aba + ba)^*(ab + b + 1) + b \\ X_3 &= a(aa + aba + ba)^*(ab + b + 1) + 1 \end{cases}$$

- Remplaçons X_3 par $aX_2 + 1$.
- Remplaçons X_1 par $(a + ba)X_2 + b$.
- Résolvons l'équation $X_2 = (aa + aba + ba)X_2 + (ab + b + 1)$.
- Reportons.
- Le langage reconnu par l'automate est donné par X_1, et donc vaut $(a + ba)(aa + aba + ba)^*(ab + b + 1) + b$.